OS-Level Implementation Details of a Raiden Game

Ritik Raj
B.Tech in CS and AI
NST, Rishihood University
230106

Prince Sahoo
B.Tech in CS and AI
NST, Rishihood University
230060

Dibyajyoti Behera
B.Tech in CS and AI
NST, Rishihood University
230052

***Abstract*—This report explains the operating system concepts we applied to build our terminal-based C game, Raiden. Because standard C libraries are often too slow or unpredictable for real-time games, we had to implement our own lower-level solutions. We break down our specific approaches to file handling, security and error trapping, process execution, non-blocking I/O, and custom memory management, showing how we optimized each area for a smooth user experience.**

***Index Terms*—Memory Pool, POSIX I/O, File Handling, State Machine, Terminal Control**

I. Methodology

To make a game run smoothly in a terminal at a high frame rate, we had to bypass standard buffered operations. Below is the breakdown of how we handled core OS responsibilities directly in the code.

A. File Systems

Instead of relying on the standard `<stdio.h>` `FILE` streams (which buffer data in ways we can't easily predict), we used direct POSIX system calls (`open`, `read`, `write`, `close`) from `<fcntl.h>` and `<unistd.h>`.

- **Data Storage:** All game data is stored in flat text files (`users.txt`, `current_user.txt`, `leaderboard.txt`). We load these files straight into a predefined memory buffer using `read()` rather than reading them line-by-line with `fgets()`.
- **File Permissions & Flags:** When creating or modifying files, we explicitly set the `0644` permission mask so standard users can read and write to them safely. We use `O_APPEND` to add new usernames without overwriting history, and `O_TRUNC` when updating the leaderboard so that old, out-of-date scores are cleanly wiped in a single atomic-style operation.

B. Security and Halt/Error Handling

When dealing with a terminal, crashing in the middle of execution can leave the user's command prompt broken (invisible cursor, weird formatting). We put several safeguards in place to handle errors and bad input.

- **Terminal Restoration:** We modify the terminal's hardware state to make the game work. If the game crashes, we need to revert those changes. We fixed this by registering a `kb_restore()` function using the C `atexit()` system call. No matter how the process halts—even if it hits a fatal memory error—the OS guarantees `kb_restore()` will run, bringing the terminal back to normal.
- **Input Validation:** In `user.c`, the username prompt is strictly controlled. We wrote a loop that physically ignores any keystroke that isn't a letter or number, and we force uppercase letters into lowercase mathmatically. This completely stops users from injecting special characters that might break our flat file system logic.
- **Buffer Overflows:** All string copies and file reads in the codebase use strict length boundaries (like `max_len - 1`). We never use unsafe functions like `strcpy` or `gets`, ensuring that corrupted save files can't overflow our arrays.

C. Process Management

We built the game as a single-threaded process. Managing multiple threads for input, rendering, and game logic would introduce race conditions and require complex mutex locks.

- **Finite State Machine (FSM):** We control the entire process flow using an FSM defined in `gamestate.h`. The main loop simply checks a variable like `STATE_TITLE` or `STATE_PLAYING` and calls the right function. This makes it impossible for two conflicting processes (like the menu and the game) to run at the same time.
- **CPU Pacing:** A standard `while(1)` loop will consume 100% of a CPU core. To fix this, we call `usleep(FRAME_US)` at the end of every frame. This voluntarily tells the OS scheduler to pause our process, handing CPU time back to the rest of the system until the exact moment we need to draw the next frame.

D. I/O Management

Standard terminal I/O waits for the user to press the "Enter" key before sending data, and it prints every character you type to the screen. That doesn't work for an action game where you need to hold the "W" key to fly forward.

- **Raw Mode:** In `keyboard.c`, we use the POSIX `termios` library to disable canonical mode (`ICANON`) and local echo (`ECHO`). We set `VMIN=0` and `VTIME=0`, which tells the OS: *"give me whatever keys are pressed right now, and if nothing is pressed, don't wait."*
- **Handling Multiple Keys:** To allow diagonal movement (pressing W and D at the same time), we wrote `kb_drain_keys()`. The OS keyboard repeat function actually sends alternating characters (w, d, w, d) very fast.

Our function loops through the entire input buffer in one frame, mapping all those characters to a single bitmask (`KeyState state = KS_UP | KS_RIGHT`).

- **Double Buffering:** We draw the screen by sending direct ANSI escape codes to `stdout`. To stop the screen from flickering, we keep two 1920-byte arrays: `front` and `back`. Every frame, we compare them. We only send print commands for the specific X, Y coordinates that actually changed, cutting our output bandwidth down from thousands of bytes per frame to just a handful.

E. Memory Management

Calling `malloc()` and `free()` during a fast-paced game is a bad idea because it forces the OS to search for free RAM, causing frame drops (latency spikes) and memory fragmentation. To solve this, we wrote a custom memory allocator in `memory.c`.

- **The Memory Pool:** When the game starts, we call `malloc()` exactly one time to grab a large, contiguous chunk of memory.
- **Custom Allocation:** When the game needs memory (like spawning a new bullet), it calls our `my_alloc()` function. `my_alloc()` searches our pre-allocated pool for a freed block using a "first-fit" search. If it doesn't find one, it just increments a pointer (a bump allocator) to give out fresh memory.
- **Zero-Overhead Tracking:** We built a `BlockHeader` struct (containing `size` and `is_free`) that we secretly place right before the memory pointer we hand back to the game. When a bullet is destroyed, we pass its pointer to `my_dealloc()`. By simply subtracting the size of the header from the pointer, we instantly find the metadata and mark it as `is_free = 1`. This requires no external tracking tables and runs in $O(1)$ constant time.

II. Future Scope

While our current implementation is fast and stable, there are a few OS-level upgrades we could make in the future. We could replace our `usleep` polling loop with asynchronous event triggers like `select()` or `epoll()`, which would let the OS wake the game up exactly when a key is pressed. Additionally, if the leaderboard gets too big, we could map `leaderboard.txt` directly into memory using `mmap()`, letting the operating system's paging hardware handle file reads automatically.

III. Conclusion

By stepping away from high-level C abstractions and interacting directly with POSIX system calls, we were able to build a highly optimized game engine. Writing our own memory pool, managing raw terminal hardware, and explicitly handling file descriptors gave us the exact control we needed to hit a stable framerate without wasting system resources.

References

[1] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Pearson, 2017.

[2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley, 2013.